

Conexión a MongoDB y consultas sobre notas de lectura

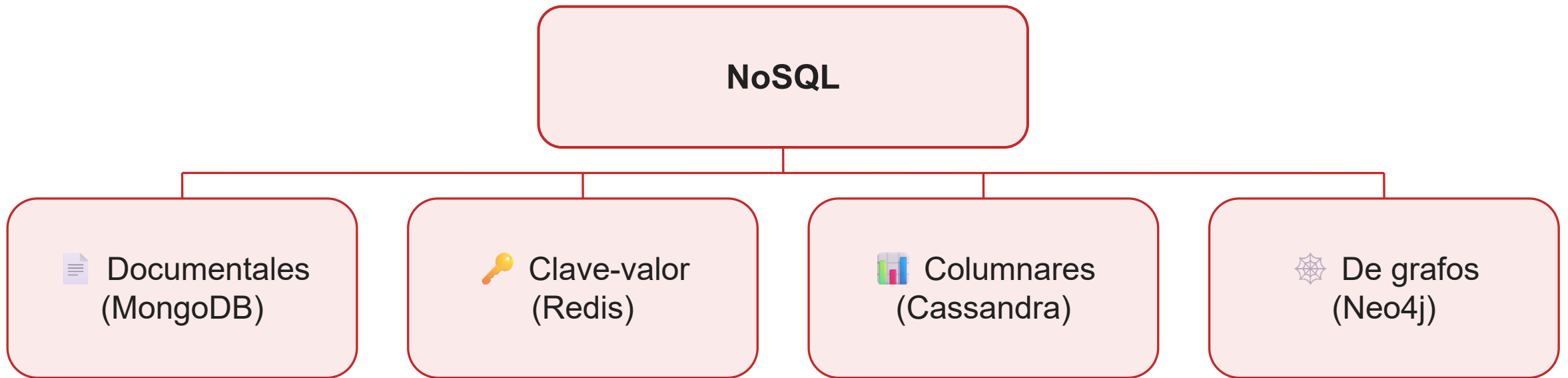
1. Conexión a MongoDB y consultas sobre notas de lectura

Conexión a MongoDB y consultas sobre notas de lectura

Hasta ahora, todo lo que has persistido ha vivido dentro de PostgreSQL — incluso el JSONB del Tema 2 seguía siendo una columna de una tabla relacional. Este tema da un salto real: una base de datos completamente distinta, sin tablas de ningún tipo.

Qué es NoSQL

NoSQL agrupa distintas familias de bases de datos que no siguen como modelo principal el relacional tradicional de tablas relacionadas mediante claves y JOIN. No es un único tipo de motor, sino varias categorías, pensada cada una para problemas distintos:



Las cuatro familias NoSQL

Familia	Qué guarda	Uso habitual
Documentales	Documentos autocontenidos, tipo JSON (este tema)	Catálogos de producto, perfiles, contenido con estructura variable
Clave-valor	Pares simples clave → valor (p. ej. Redis)	Cachés, sesiones, contadores — lecturas/escrituras muy rápidas
Columnares	Tablas distribuidas por particiones, según las consultas previstas	Grandes volúmenes distribuidos, alta disponibilidad
De grafos	Nodos y aristas — relaciones muy conectadas	Redes sociales, sistemas de recomendación

Documento y colección: las dos piezas

De las cuatro familias, este tema se centra en la documental — con MongoDB como motor concreto. Antes de verla en código, dos piezas la definen, y ninguna es una tabla:

- Documento: un objeto completo, tipo JSON, autocontenido — con su propia estructura, que puede variar de un documento a otro dentro de la misma colección.
- Colección: un grupo de documentos. Por defecto admite un esquema flexible, aunque MongoDB permite añadir reglas de validación si necesitas imponer una estructura determinada.

Dos documentos, la misma colección

Dos documentos reales dentro de la colección `nota_lectura`, con estructura distinta entre sí — el segundo tiene un campo (`comentario`) que el primero ni siquiera necesita:

```
// Colección "nota_lectura"
{ "_id": ObjectId("..."), "libroId": 1, "autor": "ana", "puntuacion": 8 }
{ "_id": ObjectId("..."), "libroId": 1, "autor": "javi", "puntuacion": 9,
  "comentario": "Me ha encantado el ritmo" }
```

Relacional, JSONB y documental

Aspecto	Relacional	JSONB (Tema 2)	Documental (este tema)
Estructura de fondo	Tablas y filas	Tabla, columna en JSON	Ninguna tabla — todo es documento
Estructura fija por defecto	Sí	Sí, salvo el JSONB	No — aunque puede añadirse validación
Claves foráneas	Sí	Sí	No (relación lógica, a mano)
JOIN	Sí	Sí	No como en SQL; existe \$lookup

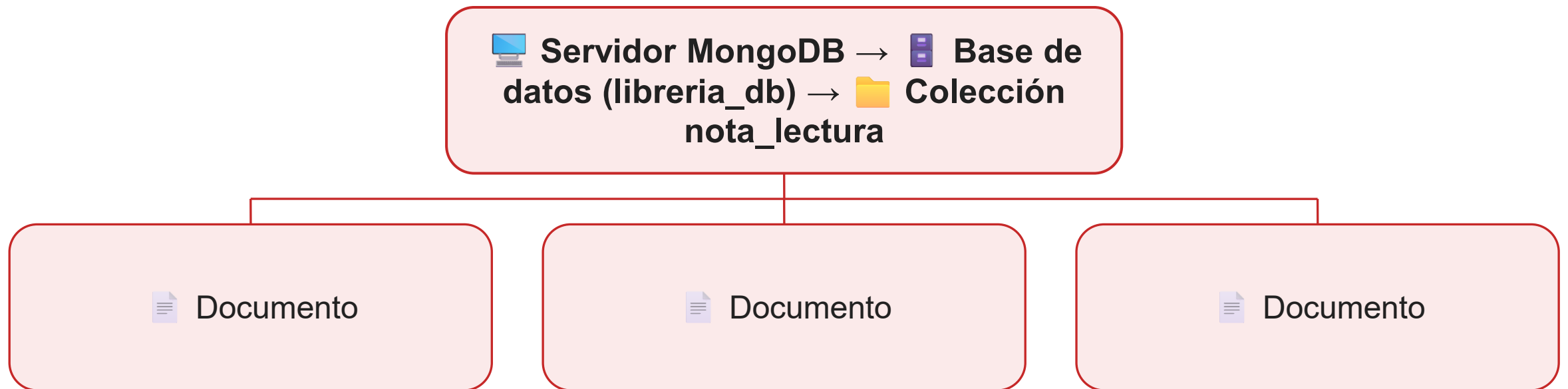
La diferencia clave con el JSONB del Tema 2: allí, el JSON era una columna dentro de una tabla relacional normal — aquí, todo es documento, no hay ninguna tabla por debajo.

Qué es MongoDB

Podríamos necesitar una base de datos documental para las notas de lectura de nuestra librería — cada nota es un documento independiente, sin relaciones fuertes entre ellas, con una forma que podría cambiar con el tiempo. El motor documental que vas a usar para eso es MongoDB.

MongoDB almacena sus documentos en BSON (una representación binaria de JSON). Todo documento tiene un campo `_id` único y, si no proporcionas uno, MongoDB genera normalmente un `ObjectId`. En Spring Data puedes representarlo con un `String`, en vez del `Long` autoincremental de PostgreSQL.

Servidor, base de datos, colección, documento



Un documento y su consulta equivalente

```
// Un documento
{ "_id": ObjectId("..."), "libroId": 1, "autor": "ana", "puntuacion": 8
}

// El equivalente a un SELECT sencillo
db.nota_lectura.find({ "libroId": 1 })
```

Vale la pena verlo en esta forma cruda antes de que Spring lo esconda tras un repositorio — mongosh es la consola de MongoDB, análoga a psql.

Cuándo encaja lo documental

Cuando los datos son autocontenidos y su estructura puede evolucionar con flexibilidad, el modelo documental encaja bien. Cuando el dominio tiene muchas relaciones entre entidades que deben mantenerse consistentes —una venta con clientes, productos y facturas—, el modelo relacional suele resultar más natural.

Esto no significa que MongoDB carezca de transacciones: las operaciones sobre un único documento son atómicas, y también admite transacciones sobre varios documentos. La diferencia está sobre todo en cómo modelas los datos y sus relaciones, no en que un motor tenga transacciones y el otro no.

Por qué dos bases de datos distintas

El catálogo (Libro/Editorial) vive en PostgreSQL — relaciones claras, transacciones fuertes. Pero los lectores pueden escribir notas de lectura: documentos independientes entre sí, sin relaciones que mantener, con forma que podría evolucionar. Eso encaja mejor en MongoDB.

Usar dos motores a la vez es una decisión de arquitectura habitual, no una moda — cada uno se usa donde encaja mejor:

La misma aplicación, dos motores



Tu aplicación Spring Boot

Spring Data JPA



PostgreSQL
Libro, Editorial

Spring Data MongoDB



MongoDB
NotaLectura

Cada motor a través de su propio Spring Data — no hay ningún puente automático entre ellos, y esa ausencia de puente es lo que resuelves tú mismo en el resto de este apartado.

Establecer la conexión: la dependencia

Antes de escribir ninguna entidad ni repositorio, tu aplicación necesita saber cómo llegar a MongoDB — igual que necesitó saber cómo llegar a PostgreSQL antes de poder usar @Entity. Una dependencia nueva en el pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

La propiedad de conexión

En application-dev.yml, en paralelo al datasource de PostgreSQL que ya conoces del Tema 1:

```
spring:
  mongodb:
    uri: mongodb://localhost:27017/libreria_db
```

La propiedad real es `spring.mongodb.uri`, no `spring.data.mongodb.uri`. Es fácil confundirse con tutoriales antiguos: en la versión de Spring Boot de este curso, `mongodb` cuelga directamente de `spring`, no de `spring.data`. Comprueba siempre tu `application-dev.yml` antes de dar por buena una propiedad.

La entidad NotaLectura

```
@Document(collection = "nota_lectura")
public class NotaLectura {

    @Id
    private String id; // Spring Data mapea aquí el ObjectId de MongoDB

    private Long libroId; // relación "lógica" con el catálogo en PostgreSQL
    private String autor;
    private Integer puntuacion;
    private String comentario;
}
```

`@Document(collection = "nota_lectura")` es el equivalente Mongo de `@Entity/@Table`. El `@Id` se declara `String` porque Spring Data puede mapear a él el `ObjectId` que MongoDB genera cuando no proporcionas un `_id`.

librold no es una clave foránea real

Aunque el campo se llame librold y apunte conceptualmente a un Libro de PostgreSQL, no hay ninguna integridad referencial automática entre dos motores de base de datos distintos. MongoDB no sabe nada de PostgreSQL, ni al revés — es responsabilidad de tu código mantener esa relación con sentido.

El repositorio, con naming de método

```
public interface NotaLecturaRepository
    extends MongoRepository<NotaLectura, String> {
    List<NotaLectura> findByLibroId(Long libroId);
}
```

Spring Data también genera consultas automáticas por el nombre del método en MongoDB, exactamente igual que hacía JpaRepository en el Tema 1 — sin que escribas una sola query explícita.

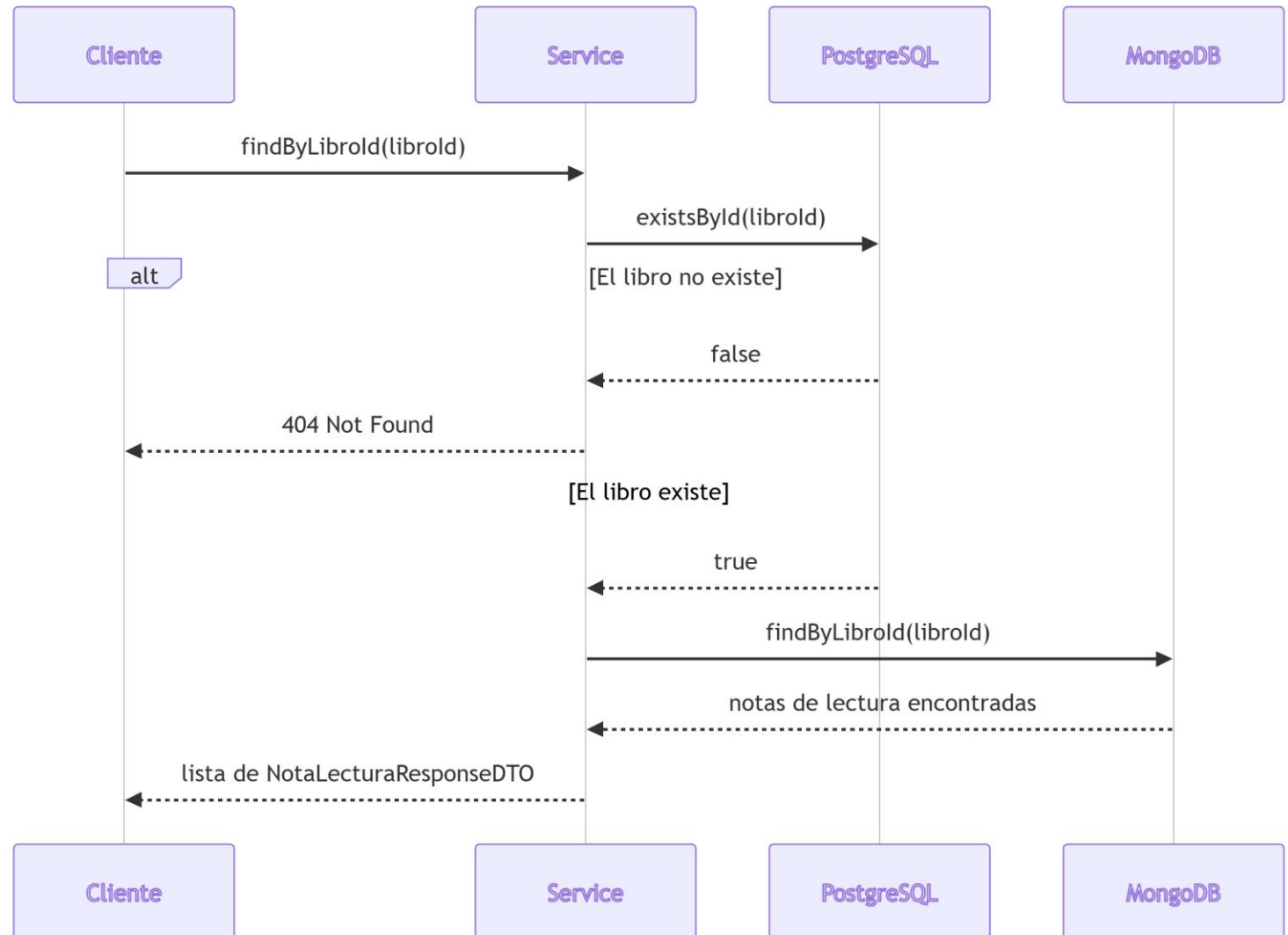
Integridad referencial manual: el código

```
public List<NotaLecturaResponseDTO> findByLibroId(Long libroId) {  
    // 1. Verificamos que el libro exista en PostgreSQL  
    if (!libroRepository.existsById(libroId)) {  
        throw new ResponseStatusException(  
            HttpStatus.NOT_FOUND, "Libro no encontrado");  
    }  
    // 2. Buscamos en MongoDB y mapeamos  
    return notaLecturaRepository.findByLibroId(libroId).stream()  
        .map(this::mapToDTO).toList();  
}
```

Como Mongo no puede validar una clave foránea hacia una tabla de PostgreSQL, el propio código de la aplicación tiene que hacerlo — comprobando primero contra PostgreSQL antes de tocar Mongo.

El flujo completo, con los dos motores

Primero PostgreSQL, y solo si responde que sí, se llega a tocar Mongo — al revés estarías haciendo una consulta de más cuando el libro no existe.



Crear una nota de lectura, con el autor protegido

El campo autor no puede venir del cliente en el cuerpo de la petición — si lo dejaras, cualquiera podría escribir el nombre de otra persona y hacerse pasar por ella. Por eso el DTO que valida lo que manda el cliente y el objeto que de verdad se usa para crear el documento no son el mismo.

Dos DTOs: lo que llega y lo que se crea

```
public record NotaLecturaRequestDTO(  
    @NotNull @Min(1) @Max(10) Integer puntuacion,  
    @NotBlank String comentario  
) {}  
  
public record NotaLecturaCreatedDTO(  
    Long libroId, String autor, Integer puntuacion, String comentario) {}
```

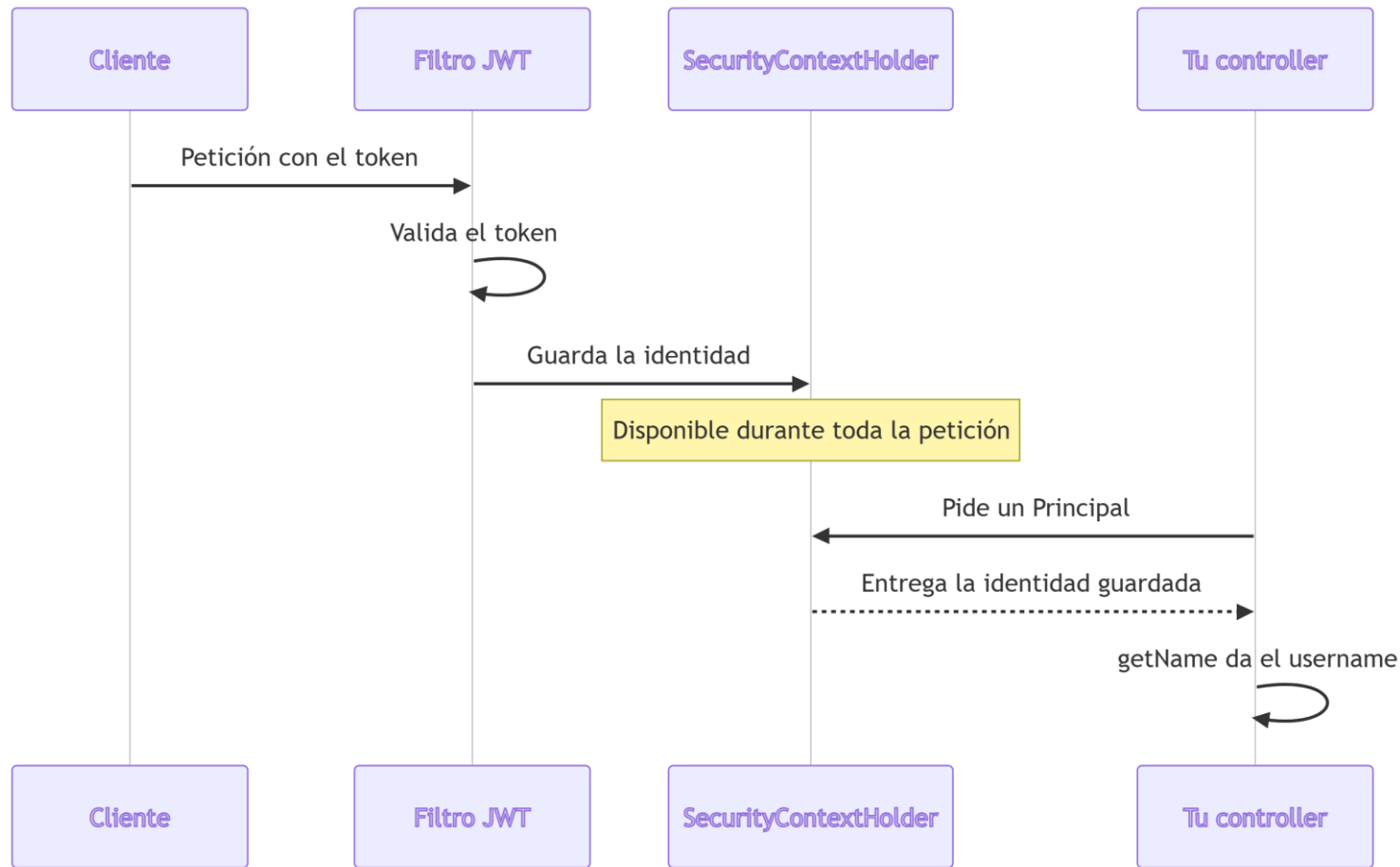
MongoDB flexible no significa API sin reglas: MongoDB admite por defecto que dos documentos tengan campos distintos, pero eso no impide que tu aplicación imponga reglas más estrictas — son dos capas diferentes: lo que admite el motor y lo que permite tu API.

Juntando los dos DTOs, en el controller

```
@PostMapping
public ResponseEntity<NotaLecturaResponseDTO> create(
    @PathVariable Long libroId,
    @Valid @RequestBody NotaLecturaRequestDTO dto,
    Principal principal
) {
    NotaLecturaCreatedDTO createdDTO = new NotaLecturaCreatedDTO(
        libroId, principal.getName(), dto.puntuacion(), dto.comentario());
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(notaLecturaService.create(createdDTO));
}
```

Principal es una interfaz estándar de Java — Spring te la entrega ya rellena como parámetro del controller, sin que tengas que pedirla ni construirla tú mismo.

De dónde sale la identidad: Principal



Por debajo es el mismo Authentication que el filtro JWT que ya construiste en PSP (Actividad 2.4) guarda al validar el token — Principal es, simplemente, esa misma identidad. `principal.getName()` da directamente el username, el mismo que ya usas en tu `/me` de PSP.

Una agregación sencilla, en memoria

```
public NotaLecturaResumenDTO getResumenByLibroId(Long libroId) {  
    List<NotaLectura> notasLectura =  
        notaLecturaRepository.findByLibroId(libroId);  
    long totalNotas = notasLectura.size();  
    double puntuacionMedia = notasLectura.stream()  
        .mapToInt(NotaLectura::getPuntuacion).average().orElse(0.0);  
    return new NotaLecturaResumenDTO(libroId, totalNotas, puntuacionMedia);  
}
```

Calcula el total y la media en memoria, con streams de Java, tras traer todos los documentos de Mongo — no usa el framework de agregación nativo de Mongo (\$group, \$avg). Es una simplificación deliberada.

En memoria o agregación nativa de Mongo

Enfoque	Cómo funciona	Cuándo conviene
En memoria (streams de Java)	Trae todos los documentos a la aplicación y calcula con <code>.stream()</code>	Pocos documentos por libro — simple de escribir y entender
Agregación de Mongo (<code>\$group</code> , <code>\$avg</code>)	El cálculo lo hace el propio motor, sin traer todos los documentos	Muchos documentos — más eficiente, consulta más compleja

Para este curso, con pocas notas de lectura por libro, la primera opción es más que suficiente.

- NoSQL agrupa distintos modelos no relacionales; las documentales (MongoDB) son una de varias familias, junto a clave-valor, columnares y de grafos.
- Un documento es un objeto autocontenido; una colección los agrupa con esquema flexible por defecto. Sin JOIN SQL, aunque existe \$lookup.
- MongoDB almacena en BSON; todo documento tiene un `_id`, normalmente un ObjectId que Spring Data puede mapear a String.

Ideas clave (cont.)

- libroid en NotaLectura es una relación lógica, no una clave foránea real — no hay integridad referencial automática entre dos motores distintos.
- El patrón de integridad referencial manual: el código comprueba en PostgreSQL antes de tocar Mongo, porque el motor no puede hacerlo por sí solo.
- La propiedad real de conexión es `spring.mongodb.uri` (no `spring.data.mongodb.uri`).